



Activitat MongoDB

Cal fer un manual tècnic:

- Portada
- Índex
- Captures de pantalla
- Codi comentat
- Webgrafia o Bibliografia utilitzada

Aquest manual s'haurà d'entregar en captures de pantalla de cada un dels apartats corresponents. El podeu fer en castellà, anglès o català.



Comencem!!!



- 1) **Instal·lació:** Instal·leu el driver oficial mitjançant la comanda `pip install pymongo`.
- 2) **Instància del Client:** Importeu `MongoClient` i creeu una instància. Recordeu configurar el paràmetre `serverSelectionTimeoutMS` (per exemple, a 2000ms) per evitar que el programa es quedi esperant indefinidament si el servidor està caigut.

```
# sempre mongodb:user//password@servidor:port/
```

- 3) **Control d'Errors:** Importeu `ConnectionFailure` del mòdul `pymongo.errors`. Utilitzeu un bloc `try-except` per capturar aquest error. Si la connexió falla, la funció hauria de retornar `None`.
- 4) **Verificació (Ping):** Dins del `try`, executeu `client.admin.command('ping')`. Si la connexió és correcta, quin valor té la clau 'ok' en el diccionari que retorna aquest comando?
- 5) **Accés a la Base de Dades:** Creeu una variable per a la base de dades: `db = client["nom_de_la_teva_bbdd"]`.
- 6) **Accés a la Col·lecció:** Finalment, accediu a la col·lecció específica: `col = db["nom_de_la_colleccio"]`. Aquest és l'objecte per poder fer el crud.

Feu aquesta llista:



```
usubd=[

    {
        "_id": 1,
        "username": "cacafina",
        "nomcomplet": "Dolores Fuertes de Barriga",
        "email": "barrigamal@insdanielblanxart.cat",
        "edat": "35",
    },

    {
        "_id": 2,
        "username": "fermin",
        "nomcomplet": "Fermín Galarga",
        "email": "yatusabes@insdanielblanxart.cat",
        "edat": "17",
    },

]
```

Recordeu que si no posen el camp `_id` manualment, cada vegada que executin el programa **es crearan duplicats** amb IDs diferents.

Inserts:

- Amb la instància `col` utilitzem el mètode `insert_many(llista_creada)` i ho guardem en una variable. Aquesta variable amb aquest propietat ens servirà per tenir un control dels registres fets.

```
print(f"Registres insertats: {len(res.inserted_ids)}")
```

- Cal importar per fer el control d'excepcions

```
from pymongo.errors import DuplicateKeyError, PyMongoError
```

Haureu de buscar que fa i com funciona cada una d'elles. I aplicar aquest control d'excepcions.

- Fes el mateix amb `insert_one(diccionari_usuari)`. Cal vigilar aquí serà `inserted_id` i no retornarà una llista!!!



Research:

- Anem a fer un cursor amb les recerques globals. Amb la instància col utilitzeu el mètode find({}). Aquest cursor cal guardar-ho en una variable que s'haurà d'iterar.
- Una manera de saber si hi ha resultats o no és posant aquest codi:

```
if col.count_documents({})==0:  
    print("Error no hi ha llista que mostrar")  
    return
```

- Anem a fer un diccionari amb un únic resultat. Amb la instància col utilitzeu el mètode find_one({"camp":valor}). Aquí el control de resultat és que si no el localitza retorna None. Al retornar un diccionari ja sabeu com accedir els seus valors.
- (Abans cal fer un update amb l'edat) Farem el mateix però amb edat. Penseu que aquí la recerca funciona d'aquesta manera:

```
usuedat=col.find({"edat":{"$gt":edat}})
```

, per tant, ja sabeu que le, lt, ge i gt serveixen per fer control de valors numèrics. Cal l'operador \$!!!.

Update:

- Aquí hem de pensar que cal buscar escollir camp per trobar el document i camp que volem actualitzar o camps que volem actualitzar.
- Recuperem excepció genèrica de Mongo. Pot passar que tinguem un error en la connexió.
- És important tenir aquest element dins del diccionari que volem modificar, així no esborrem els camps que no volem modificar.

```
canvis={"$set":{  
  
}}
```

- Amb el mètode i la instància realitzem que guardem en una variable.
<res=col.update_one(filtre, canvis)> Realitzarem la modificació de l'edat d'un dels nostres usuaris. Tenim un valor amb string i el passarem a int. Així podrem fer el research.
- Amb la propietat res.matched_count podem saber si s'ha trobat el registre o no. Retorna



un int.

- Amb la propietat `res. modified_count` podem saber quants elements s'han modificat. Retorna un int.

Delete:

- Aquí no cal pensar quin camp utilitzar, id.
- Guardem amb una variable el resultat de fer `<col.delete_one({'_id':id})>`.
- I amb aquesta resposta i la propietat `deleted_count`, que retorna un valor numèric, podem saber si s'ha esborrat o no el registre.
- Recuperem excepció genèrica per si error de connexió.